

Completion-Time Feedback for Prefill Budgets: A Controlled Single-GPU Serving Study

Session D — independent experimental report

4 October 2026

Abstract

We implement an aggregate prefill-token cap in native single-GPU serving, preserving request order, admission rules, model precision and kernels. Real mixed request traces show a repeatable fixed-budget throughput–generation-stall tradeoff. In the development trace, increasing the cap from 512 to 2,048 improves output throughput by 6.6%, while increasing request maximum-gap P95. A four-level controller using only the previous completed mixed-step duration mostly selects one level at moderate load. We compare it with fixed budgets, a decode-count rule, its dominant fixed level and a matched-mean cap, including a normal-memory PRO 6000 pressure extension. The frozen moderate-load confirmation favors a fixed cap. At high load, feedback has a small exploratory joint-goodput advantage over the tested constants, with lower raw throughput and longer tail waiting. The contribution is a reproducible, bounded empirical result rather than a claim that adaptive chunking is new or broadly beneficial.

1 Question and scope

Long prompt processing shares execution time with ongoing token generation in continuous batching. We test whether changing only the aggregate number of prompt tokens allowed per iteration improves complete-service outcomes over a tuned, deployable fixed cap. Equal token counts need not imply equal iteration costs, but this is a hypothesis to test, not a premise establishing adaptive superiority. This report distinguishes an observed throughput–stall tradeoff, an informative state signal, and an end-to-end improvement: none implies the next automatically.

2 Runtime intervention

An initial launch failed before any request was executed because an environment default overrode the requested in-process mode; it is retained as a zero-measurement startup failure. Explicitly disabling the default multiprocessing environment flag restores the requested in-process configuration. The runtime is vLLM 0.26.0, serving OLMoE-1B-7B-0924-Instruct in BF16 on one NVIDIA RTX PRO 6000 Blackwell Server Edition (97,887 MiB reported memory). The engine has a fixed 4,096 total-token limit, 192 sequence slots and 4,096 maximum context length. The normal-capacity engine uses the runtime memory profiler with a 90% device-memory target; no manual small-KV limit is imposed. Resolved KV capacity is recorded. Prefix caching, speculative decoding and asynchronous scheduling are disabled for every arm. The FCFS admission/allocation logic, victim selection, model weights, precision and operators are common to all arms. The first experiment is resident serving with no KV connector; zero preemptions are checked, so recovery behavior is outside the evaluated regime.

A process-local scheduler patch introduces one counter P per iteration. For the running and waiting request loops, it clips only work beyond the remaining prompt-token allowance. The counter is decremented only after successful native allocation. Waiting requests use the native computed-prefix position; running requests use their current position. The patch neither sorts requests nor changes decode priority. At runtime we check prompt work $\leq P$, no preemption, and exactly one scheduled token for every decoder already resident before the step. The original installed scheduler is not modified.

The candidate action set is $P \in \{256, 512, 1024, 2048\}$. Additional fixed controls use 1,072 to match its development mean cap and 3,840 as a stronger upper-budget endpoint; both receive dedicated full-trace warmups. Fixed arms use one cap for the whole service trace. The count-only rule uses 2,048 below eight active decoders and 512 otherwise. The feedback rule uses the same levels and adds the elapsed time of the most recent completed mixed step. Starting at 1,024, it decreases one level when the step exceeds a target $T = 24$ ms, increases one level below $0.7T$, and otherwise holds. With no active decoder it uses 2,048. The

target is chosen after the first fixed comparison, whose mixed steps span roughly 14–28 ms, and is frozen before the long-document holdout. Only completed observations are used; no expert routing, extra forward pass, learned model, full shape scan, or synchronization is added. The positive minimum cap and the finite workload prevent indefinite withholding of prefill service in this domain. This is not a worst-case wall-clock guarantee under arbitrary arrivals.

3 Experimental method

Each development trace has 36 requests: 12 short GSM8K prompts with 384 generated tokens each and 24 long QMSum prompts with 160 tokens each. Eight short requests arrive at zero, four at 3 seconds. Long requests arrive in three waves beginning at 0.6, 2.5 and 5 seconds, with eight requests per wave and 40 ms within-wave spacing. Arrivals are a common absolute clock, not completions from another policy. The development prompt total is 92,625 tokens and every arm must generate 8,448 tokens and drain all requests. EOS is ignored for this controlled scheduling experiment; this does not evaluate natural termination or task quality. The PRO 6000 extension also includes 12-request low-load and 160-request high-load traces. The high-load trace combines 32 short requests and 128 long prompts with 512 output tokens each (within the model context limit), totaling 471,895 prompt tokens and 77,824 output tokens. Actual resident count and KV occupancy are measured; the label high load alone does not prove a capacity bottleneck. A separate long-document source trace replaces the 24 QMSum prompts; the short background prompts are shared and therefore this is not a fully disjoint holdout.

All four cap levels run the full development trace as warmup before measurement, using one common engine and compilation domain. Request state drains between episodes and prefix caching is disabled. The first fixed comparison is ordered 512, 2,048, 2,048, 512. The 1,024 and 256 fixed controls and both dynamic rules are then compared in forward and reverse orders. Fixed 2,048 is selected by development joint goodput before holdout. The later 3,840 endpoint is an explicitly labeled additional exploratory control. Later selection and confirmation steps are identified separately in the results. A common host lock protects the complete campaign; the controller verifies no other GPU process before initialization.

Every step records actual prompt/decode work, pre-step decoder count and summed decoder context, policy selection overhead, scheduler time, and the elapsed `engine.step()` envelope. This envelope includes host and device work through returned outputs; it is not a CUDA-event kernel time. There is no added device synchronization. Every request retains planned arrival, actual submission, first scheduling, completion, output IDs, and every host-observed output time.

Throughput uses all generated tokens (8,448 in development) divided by the full trace duration, including waiting and drain. TTFT and completion latency are relative to planned arrival. For each request, maximum generation gap is the maximum difference between consecutive observed output timestamps, excluding TTFT. Repeated timestamps are reported rather than interpreted as observable intra-return token timing. The primary joint SLO requires $\text{TTFT} \leq 4$ s, maximum generation gap ≤ 100 ms, and completion latency ≤ 20 s. Request goodput is the count passing all three divided by full trace duration; output goodput counts only outputs of passing requests. These thresholds are fixed before observing results and describe an experimental contract, not a claimed production requirement.

4 Results

All 57 measured episodes completed, covering 4,636 request completions and 1,974,016 generated tokens. No measured request is dropped from the denominator. All token accounting checks pass.

4.1 The fixed-budget tradeoff is real

On the development trace, fixed 2,048 improves output throughput by 6.60% relative to 512. Request maximum-gap P95 rises from 19.10 to 28.69 ms. Both satisfy the same primary joint SLO for every request. Thus a finer gap tradeoff exists even where the declared SLO does not reward smaller chunks.

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 256	2	951.1	1.454	3.942	17.34	4.053
Fixed 512	2	1037.0	0.651	3.492	19.10	4.419
Fixed 1024	2	1085.7	0.266	3.244	21.45	4.626
Fixed 2048	2	1105.4	0.104	3.134	28.69	4.711
Decode count	2	1036.8	0.650	3.457	22.80	4.418
Feedback	2	1087.5	0.254	3.221	26.83	4.634

Table 1: Development trace; fixed selection and candidate construction use these observations. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

4.2 Frozen long-document confirmation

On the long-document holdout, feedback changes throughput by -0.89% and joint goodput by -0.89% versus the development-selected fixed 2,048. The short background prompts are shared with development. The three main-policy repetitions provide repeatability evidence, not a broad workload-population confidence interval.

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 2048	3	1103.6	0.101	3.146	28.44	4.703
Fixed 1024	2	1092.7	0.251	3.212	21.56	4.656
Decode count	3	1042.2	0.616	3.436	22.69	4.441
Feedback	3	1093.8	0.244	3.193	25.99	4.661

Table 2: Frozen long-document holdout. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

4.3 Normal-memory pressure extension

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 1024	2	2114.7	11.349	25.913	66.74	0.652
Fixed 2048	2	2232.1	4.760	27.263	70.54	0.688
Decode count	2	1867.6	22.391	25.228	44.37	1.092
Feedback	2	1600.9	32.452	25.438	32.07	1.121

Table 3: High load: 160 requests, normal memory allocation. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 2048	4	476.1	0.118	1.786	25.15	2.029
Feedback	2	476.5	0.158	1.806	24.87	2.030

Table 4: Low load: 12 requests. Decode-count and fixed 2,048 choose exactly the same cap at every step and are merged here. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

4.4 Dominant-level and mean-budget controls

In development, feedback selects 1,024 in 84 of 88 mixed steps in each repetition, with four selections of 2,048. Its mean mixed cap is 1,070.55. Fixed 1,024 is the direct dominant-level control; fixed 1,072 additionally approximates the mean allowance. This comparison counts full realized service time, rather than assuming identical state trajectories from similar average caps.

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 1072	2	1093.5	0.239	3.214	21.81	4.660
Fixed 3840	2	1103.3	0.057	3.152	39.69	4.701
Feedback	2	1087.0	0.256	3.227	26.95	4.632

Table 5: Additional exploratory controls: matched mean cap and upper fixed endpoint. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

Policy	n	Output/s tokens	TTFT P95 seconds	Mean flow seconds	Gap P95 ms	Goodput/s requests
Fixed 256	2	1513.5	35.271	27.227	28.36	0.778
Fixed 304	2	1622.1	31.387	26.117	38.50	0.896
Fixed 384	2	1748.5	26.934	25.315	35.46	1.078
Fixed 512	2	1858.2	22.577	25.437	77.30	0.825
Fixed 768	2	2028.5	15.724	25.455	61.65	0.834
Fixed 3840	2	2273.5	2.058	28.029	83.14	0.000
Feedback	2	1605.9	32.336	25.381	32.68	1.125

Table 6: Additional high-load fixed controls; constants apply to the entire trace, never individual segments. Values are means of complete-run statistics; Gap P95 is across per-request maximum observed gaps, not pooled token gaps.

Across measured episodes, peak resident concurrency is 160 and peak observed KV occupancy is 88.31%. All measured preemption counts are zero. The largest per-run P95 policy-choice time is 3.56 microseconds; this excludes common observation construction, which remains included in all end-to-end times. No second model is introduced because an independently useful extra signal has not yet been established.

4.5 A conditional gain, not a uniform improvement

In the additional high-load comparison, feedback reaches 1.125 qualified requests/s versus 1.078 for fixed 384, the highest mean among the tested high-load fixed constants. This is 4.28% higher goodput, alongside 8.15% lower raw throughput, 20.06% higher TTFT P95, and 9.52% higher completion-latency P95. This is a measured tradeoff, not Pareto dominance. Fixed 304 approximates feedback’s 303.42-token realized mixed-step mean and 304.03-token mean cap in this supplementary group, but has only 0.896 qualified requests/s. Mean work alone does not explain the high-load goodput difference.

The original high-load main group has zero gap-SLO failures. Feedback admits 30–31 first-wave long requests into the joint SLO versus 21–22 for the count rule; later waves fail under both. Compared with count-only scheduling, feedback improves complete-request mean outcomes for 87 of 160 request IDs and worsens 73; the joint-goodput increment comes only from early requests crossing thresholds. Third-wave mean completion rises from 34.57 to 39.62 seconds. This explicitly exposes the service cost of the gain.

All fixed-512 supplementary trials remain in the report: its goodput ranges from 0.571 to 1.078 requests/s. In the slower-SLO trial, an observed 108.97 ms gap causes 67 gap failures; the associated step envelope is 108.47 ms and scheduler time is 1.07 ms. This record is neither removed nor attributed to a specific GPU or host cause. Fixed-3840 goodput is zero because all 160 requests exceed the 20-second completion constraint, despite the largest raw throughput; none fails its gap condition. These results show sensitivity to the full SLO conjunction, not only average step latency.

The complete retained log contains one monitored Triton JIT warning during the initial 512 warmup and no recorded JIT compilation warning in the measured episodes. This is a log-based check, not a claim that every source of runtime jitter is observable.

4.6 What the observed state explains

At exactly 512 actual prompt tokens, mean completed-step time rises from 13.74 to 17.81 ms when the decoder-count bin changes from 1–7 to 16–23; the reverse repetition gives 14.07 to 17.93 ms. At 2,048 actual prompt tokens, the 8–15 versus 16–23 bins give 24.92 versus 27.38 ms, reproduced as 24.93 versus 27.37 ms. These comparisons hold prompt work fixed, not total tokens. For a stricter fixed-total comparison, two fixed-1,024 runs each contain 19 steps with exactly 10 decoders. Splitting them by context sum gives mean

costs 18.268 versus 18.730 ms and 18.281 versus 18.741 ms. This is a small repeatable association at 1,034 total scheduled tokens. A five-step bin with 18 decoders has the opposite direction, so we do not infer a monotonic causal law or a validated independent prediction signal.

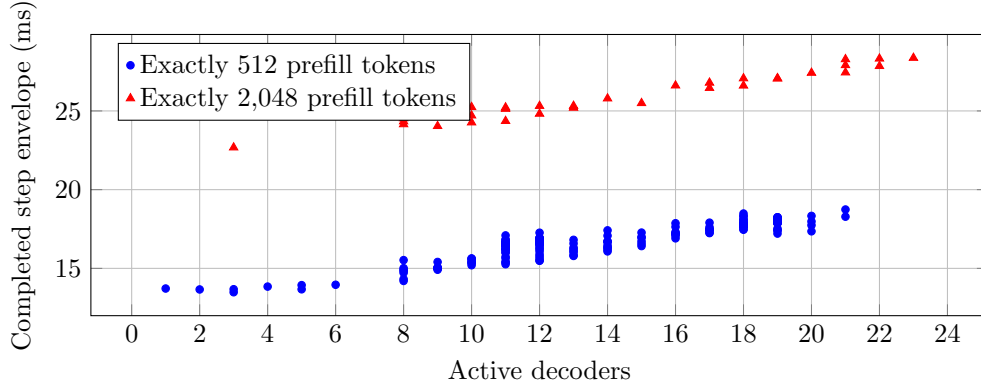


Figure 1: Actual mixed steps from the first measured fixed pair. Partial tail chunks are excluded only from this equal-prompt-work diagnostic, not from service metrics.

The fixed-1,024 and feedback development runs agree on the complete output sequence for 24 of 36 requests; each policy agrees on all 36 sequences between its own repetitions. Every run still generates exactly 8,448 tokens. This is consistent with shape-dependent numerical execution changing greedy trajectories; it does not establish either a precision change or a task-quality difference.

5 Related work and interpretation

Sarathi-Serve establishes chunked prefills and decode-maximal batches as tools for the throughput–latency tradeoff [1]. Its current official implementation also supports decreasing chunk sizes with progress through a prompt [2]. QoSServe chooses chunks using deadline state and predicted execution cost from request count and context information [3]. These are direct prior art, so neither adaptive chunking nor context awareness is claimed here as new. DLFP is particularly close: it uses proportional latency feedback to bound overlapping prefill, and reports both small-model gains and failed transfer to larger configurations [4]. Its asynchronous scheduler-cadence measurement and unsuccessful completion-aware revision make measurement fidelity and strong fixed controls central to this experiment. A synchronous host completion envelope is a practical observation choice here, not by itself a novelty claim.

6 Limitations and conclusion

These experiments establish the fixed-budget tradeoff and a conditional high-load goodput gain, but not a consistent improvement across full service metrics and loads. The frozen long-document confirmation does not beat its development-selected fixed policy. At moderate load, most actions collapse to the same cap; at higher load, the feedback target can be incompatible with the decode cost already present, so reducing prompt service can move delay into the waiting queue. The scope is one model, one GPU, in-process synchronous vLLM, fixed output lengths, and a small set of structured arrival traces. Model precision is unchanged, but different batch shapes can alter greedy outputs through floating-point execution; fixed output lengths hold service work constant and are not a semantic-equivalence claim. Host-observed delivery gaps omit network transport and do not isolate device-only latency. GPU clocks are not locked. A 100 ms gap SLO leaves many small-load requests qualified under every policy, so its goodput is often determined by full service duration. No hindsight choice of a fixed cap per trace segment is presented as an online algorithm. The high-load trace was used for fixed-budget tuning and is exploratory rather than an independent high-pressure holdout. The count-only comparator is a conservative simple rule, not a tuned count-based controller. Thus the value of elapsed time beyond a strong count rule remains unresolved; a MoE label supplies no novelty claim.

Artifact and reproduction

The accompanying artifact contains the process-local scheduler intervention, the serving driver, frozen tokenized workloads, exact executed phase commands, per-step and per-request originals, numerical analysis, and this paper’s sources. The **reproduction/** directory is a clean execution bundle: its command files are reconstructed from the retained executed protocols, including the warmup-only phases. The repository README gives the launch and analysis commands. Replaying the campaign requires the recorded vLLM/Torch environment and the model weights; the artifact does not package weights or credentials.

The resolved stack is Torch 2.11.0+cu130, CUDA 13.0, vLLM 0.26.0 and NVIDIA driver 595.71.05. The model revision is 7f1c97f440f06ce36705e4f2b843edb5925f4498; all three weight-shard SHA-256 digests match the retained model manifest. Native and patched scheduler digests are recorded in the initial phase’s **patch.json**. The memory profiler allocated 71.81 GiB of KV cache, with 36,765 blocks of 16 tokens (one block is reserved). This is normal device-capacity allocation, not a reduced-cache mechanism experiment.

The analyzer validates request count, completion, fixed output length, prompt and decode token conservation, timestamp counts, and prefill budget compliance. The report builder excludes warmups and the failed zero-request startup from its 57 measured episodes. All 108 archived remote originals were verified byte-for-byte after transfer. The campaign released its shared GPU lock after the last comparison; analysis and document preparation continued locally.

References

- [1] A. Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI, 2024. <https://arxiv.org/abs/2403.02310>
- [2] Microsoft. Sarathi-Serve scheduler source, accessed 4 October 2026. https://github.com/microsoft/sarathi-serve/blob/main/sarathi/core/scheduler/sarathi_scheduler.py
- [3] QoServe: Breaking the Silos of LLM Inference Serving. ASPLOS, 2026. https://www.microsoft.com/en-us/research/wp-content/uploads/2025/10/QoServe_CC_final_v2.pdf
- [4] G. Agarwal, A. Garg and I. Singhal. Decode-Latency Feedback Prefill: A Model-Free Controller and Its Generalization Limits. 2026. <https://arxiv.org/abs/2609.38386>